Description Logic

Objects

- Object can be:
  - Physical like houses and people
  - Conceptual like courses and trips
  - abstractions like events and relations.
- Each of these types of object has its own *parts* :
  - physical (roof, doors, rooms, fixtures, etc.; arms, torso, head, etc.)
  - abstract (course title, teacher, students, meeting time, etc.; destination, conveyance, departure date, etc.)
- The parts are constrained in various ways:
  - The roof has to be connected to the walls in a certain way
  - the departure date and the first leg of a trip have to be related
- Constraints/relations:
  - Registration procedure that connects a student to a course
  - The procedure for reserving an airline seat that connects the second leg of a trip to the first

Objects

- Objects naturally fall into categories (e.g., my pet is a dog, my wife is a physician), but are very often thought of as being members of multiple categories (e.g., I am an author, an employee, and a father);
- Categories can be more general or more specific than others (e.g., Westie and Schnauzer are types of dogs, a rheumatologist is a type of physician, a father is a type of parent);
- in addition to generalization being common for categories with simple names, it is also natural for those with more complex descriptions (e.g., a part-time employee is an employee, a Canadian family with at least one child is a family, a family with three children is a family that is not childless);
- objects have parts, sometimes in multiples (e.g., books have titles, tables have legs, automobiles have wheels);
- the relationships among an object's parts is essential to its being considered a member of a category (e.g., a stack of bricks is not the same as a pile of the very same bricks).

Descriptions

- Recall FOL, we focused mainly on sentences, because sentences that express what is known. Here, we want to talk about noun phrases.
- Noun phrases add extra properties/definitions to noun.
- We represented categories of objects with one-place predicates using common nouns like Company(x), Knife(x), and Contract(x).
- if Child(x, y) and FatherOfOnlyGirls(x) were both true, y would have to be a girl.
- Note that this would be so even if we had a simple name that served as an abbreviation for a concept like this, which is very often the case in natural language (e.g., Teenager is synonymous with PersonWithAgeBetween13and19).
- Such nouns are known as category nouns such as:Hunter, Teenager, and Girl .
- there are relational nouns like Child and Age that describe objects that are parts or attributes or properties of other objects.
- Description logics (DLs) are a family of formal knowledge representation languages used to structure, manage, and reason about domain knowledge, acting as the foundation for modern ontologies and the Web Ontology Language (OWL).

A DESCRIPTION LANGUAGE (DL)

- DL has two types of symbols:
  - logical symbols, which have a fixed meaning or use,
  - Non-logical symbols, which are application-dependent.
- Types of logical symbols in DL:
  - punctuation: “[,” “],” “(,” “)”;
  - positive integers: 1, 2, 3, etc.;
  - concept-forming operators: “ALL,” “EXISTS,” “FILLS,” “AND”;
  - connectives: $\subseteq$, $=$, $\rightarrow$
- Types of non-logical symbols in DL:
  - Atomic concepts, written in capitalized mixed case, e.g., Person, WhiteWine, FatherOfOnlyGirls; DL also has a special atomic concept,Thing;
  - roles, written like atomic concepts, but preceded by “:,” e.g., :Child, :Height, :Employer, :Arm;
  - constants, written in un-capitalized mixed case, e.g., desk13, maryAnnJones.

A DESCRIPTION LANGUAGE (DL)

- Types of legal syntactic expressions in DL:
  - constants, roles, concepts, and sentences. We use $\boldsymbol{c}$ and $\boldsymbol{r}$ to range over constants and roles, respectively, $\boldsymbol{d}$ and $\boldsymbol{e}$ to range over concepts, and $\boldsymbol{a}$ to range over atomic concepts. The set of concepts of DL is the least set satisfying the following:
    - every atomic concept is a concept;
    - if r is a role and d is a concept, then $[\boldsymbol{ALL}\ r\ d]$ is a concept;
    - if r is a role and n is a positive integer, then $[\boldsymbol{EXISTS}\ n\ r]$ is a concept;
    - if r is a role and c is a constant, then $[\boldsymbol{FILLS}\ r\ c]$ is a concept;
    - if $d1 \ldots dn$ are concepts, then $[\boldsymbol{AND}\ d1 \ldots dn]$ is a concept.
- There are three types of sentences in DL:
  - if d1 and d2 are concepts, then (d1 $\sqsubseteq$ d2) is a sentence;
  - if d1 and d2 are concepts, then (d1 =. d2) is a sentence;
  - if c is a constant and d is a concept, then (c $\rightarrow$ d) is a sentence.

A DESCRIPTION LANGUAGE (DL)

- every atomic concept is a concept:
  - Atomic concept: a basic class name like Person, Student, Car.
  - **Meaning:** an atomic concept denotes a **set of individuals**.
  - This rule says: any such name is already a valid DL concept.
  - Example: Student = the set of all students.
- if r is a role and d is a concept, then [ALL r d] is a concept
  - **Role** = a binary relation / property like hasChild, teaches, partOf.
  - [ALL r d] corresponds to the universal restriction $\forall$ r . d.
  - **Meaning:** individuals whose all r-successors are in concept d.
  - **Example:** [ALL hasChild Student] = people **all of whose children** are students.
  - If someone has **no** children, this is usually **vacuously true** in standard DL semantics.

A DESCRIPTION LANGUAGE (DL)

- if r is a role and n is a positive integer, then [EXISTS n r] is a concept
  - This is a **number restriction** (specifically "at least n"). It corresponds to **≥ n r** (often written ∃≥n r or (≥ n r . ⊤)).
  - **Meaning:** individuals that have **at least n distinct r-related individuals**.
  - **Example:**[EXISTS 2 hasChild] = people with at least two children.
- if r is a role and c is a constant, then [FILLS r c] is a concept
  - This is a **has-value restriction** (often written **∃ r . {c}** or **r value c**).
  - **Meaning:** individuals that are connected by role r to the specific named individual c.
  - Example:[FILLS hasAdvisor DrFaheem]= students whose advisor is exactly DrFaheem.

A DESCRIPTION LANGUAGE (DL)

- if d1 … dn are concepts, then [AND d1 … dn] is a concept
  - This is **concept conjunction**, i.e., intersection **d1** ⊓ … ⊓ **dn**.
  - [AND d1 … dn]=d1∩⋯∩dn
  - **Meaning:** individuals that belong to **all** of the listed concepts.
  - **Example:**[AND Student Employee] = individuals who are both students and employees.
- [AND Person [ALL hasChild Student] [EXISTS 2 hasChild]]
  - A **Person** who has **at least 2 children**, and **all** of their children are **Students**.

Concept Constructors

- Conjunction (AND) $\sqcap$
  - GraduateStudent $\equiv$ Student $\sqcap$ DoesResearch
  - A GraduateStudent is a Student who also does Research
  - x must be BOTH a Student AND do Research to be a GraduateStudent.
- Disjunction (OR) $\sqcup$
  - AcademicPerson $\equiv$ Student $\sqcup$ Professor
  - An AcademicPerson is either a Student or a Professor
- Negation (NOT) $\neg$
  - NonStudent $\equiv$ Person $\sqcap \neg$Student
  - A NonStudent is a person who is not a Student

Concept Constructors

- Existential Restriction $\exists$
  - $Student \equiv \exists\, enrolledIn.Course$
  - A Student is someone who is **enrolled in at least one** Course
  - There EXISTS at least one course that the student is enrolled in.
- Universal Restriction $\forall$
  - $FullTimeProfessor \sqsubseteq \forall\, taughtBy.GraduateCourse$
  - A FullTimeProfessor teaches only graduate courses
  - ALL courses taught by them must be GraduateCourses.
- Number Restrictions $\geq$ n and $\leq n$
  - $PhDStudent \equiv Student \sqcap \geq 1\, hasAdvisor.Professor \sqcap \leq 2\, hasAdvisor.Professor$
  - A PhD student has at least 1 and at most 2 advisors

Concept Constructors

- NightStudent ≡ Student ⊓ ∃ enrolledIn.EveningCourse
  - A NightStudent is a Student who is enrolled in at least one Evening Course.
- IsolatedPatient ≡ Patient ⊓ ¬ ∃ treatedBy.Physician
  - An IsolatedPatient is a Patient who is not being treated by any Physician.
- BusyProfessor ≡ Professor ⊓ ≥4 teaches.Course
  - A BusyProfessor is a Professor who teaches at least four Courses.
- PureResearcher ≡ AcademicPerson ⊓ ¬Professor ⊓ ∃ worksOn.ResearchProject
  - A PureResearcher is an Academic Person who is not a Professor but is actively working on at least one Research Project.
- TrustedEmployee ≡ Employee ⊓ ∀ manages.CertifiedProject
  - A TrustedEmployee is an Employee who manages only Certified Projects.

Concept Constructors

- Define ***WorkingStudent*** as a person who is both a Student and an Employee.
  - WorkingStudent ≡ Student ⊓ Employee
- Define ***AcademicStaff*** as anyone who is either a Professor or a Lecturer or a Researcher.
  - AcademicStaff ≡ Professor ⊔ Lecturer ⊔ Researcher
- Define ***UndergraduateStudent*** as a Student who is not a GraduateStudent.
  - UndergraduateStudent ≡ Student ⊓ ¬GraduateStudent
- Define ***SupervisedStudent*** as a Student who has at least one Supervisor who is a Professor.
  - SupervisedStudent ≡ Student ⊓ ∃ hasSupervisor.Professor
- Define ***Polyglot*** as a Person who speaks at least three languages.
  - Polyglot ≡ Person ⊓ ≥3 speaks.Language

TBox

- The TBox defines the terminology (schema) of domain using two types of axioms:
  - Concept Inclusion (Subclass Axiom) $\sqsubseteq$
    - $A \sqsubseteq B$ (One-way)
    - "Every A is a B" (necessary condition)
    - Student $\sqsubseteq$ Person
    - Professor $\sqsubseteq$ Person $\sqcap$ $\exists$ teaches.Course
      - if something is a Professor, it MUST be a Person AND must teach at least one Course
  - Concept Equivalence (Definition) $\equiv$
    - $A \equiv B$ (Both-way)
    - A is EXACTLY B (necessary AND sufficient condition)
    - Mother $\equiv$ Woman $\sqcap$ $\exists$ hasChild.Person
      - something is a Mother IF AND ONLY IF it is a Woman who has at least one child who is a Person.

TBox

	⊑ (Subclass)	≡ (Equivalence)
Direction	One-way	Both-way
Reasoning power	Primitive concept	Defined concept
Example	Dog ⊑ Animal	Bachelor ≡ Man ⊓ ¬Married
Classifier can	Check membership	Automatically classify individuals

- Only with ≡, the reasoner can automatically classify an individual. Using just ⊑, must explicitly declare membership.

TBox

- Role Axioms in Tbox
  - Role inclusion (sub-role)
    - hasMother $\sqsubseteq$ hasParent (if X hasMother Y, then X hasParent Y)
  - Role transitivity
    - Trans(hasAncestor) (if A hasAncestor B and B hasAncestor C $\rightarrow$ A hasAncestor C)
  - Role inverse
    - hasParent $\equiv$ hasChild (hasParent is the inverse of hasChild)
  - Role disjointness
    - isMother $\sqsubseteq \neg$isFather (nothing can be both mother and father)

Mini Example

TBox

$PhDStudent \equiv Student \sqcap \exists hasAdvisor.Professor$

$GradStudent \equiv Student$

Query: $PhDStudent \sqsubseteq GradStudent$?

Answer: YES , because PhDStudent requires being a Student, which is exactly GradStudent

ABox

- The **ABox** contains facts about specific named individuals.
  - ali : Student
    - ali is a Student
  - dr_khan : Professor
    - dr_khan is a Professor
  - cs101 : Course
    - cs101 is a Course
- Role Assertions
  - enrolledIn(ali, cs101) (ali is enrolled in cs101)
  - taughtBy(cs101, dr_khan) (cs101 is taught by dr_khan)
  - hasAdvisor(ali, dr_khan) (ali's advisor is dr_khan)

ABox

- Unique Name Assumption (UNA)
  - In DL (and OWL), by default, **different names may refer to the same individual** unless explicitly stated otherwise.
  - Without UNA, reasoner might assume:
    - ali = dr_khan (unless you state: $ali \neq dr\_khan$)
  - To enforce distinct individuals: $ali \neq dr\_khan$
- Open World Assumption (OWA)
  - DL uses **Open World Assumption**: if something is not stated, it is **unknown** (not false).
    - teaches(dr_khan, cs101)
    - **Query:** Does dr_khan teach cs201?
    - **OWA answer:** UNKNOWN (not asserted, but not denied either)
    - **Compare to databases (Closed World):** answer would be NO
    - databases assume "not stored = false", but DL assumes "not stored = unknown."

Mini Example

TBox

$\text{Doctor} \equiv \text{Person} \sqcap \exists \text{treats.Patient} \sqcap \exists \text{worksIn.Hospital}$

$\text{Specialist} \equiv \text{Doctor} \sqcap \geq 1\ \text{hasSpecialization.MedicalField}$

$\text{GeneralPractitioner} \equiv \text{Doctor} \sqcap \neg \text{Specialist}$

ABox

dr_ahmed : Doctor
treats(dr_ahmed, patient_sara)
patient_sara : Patient
worksIn(dr_ahmed, city_hospital)
city_hospital : Hospital

- **What a reasoner infers automatically:**
  - dr_ahmed is a Person (because Doctor $\sqsubseteq$ Person)
  - dr_ahmed satisfies all Doctor conditions
  - patient_sara is treated by a Doctor